

A 3D rendering of the text "DL for ENG" in a bold, gold, sans-serif font. The letters are floating on a deep blue ocean with visible ripples and small waves. The sky above is a mix of light blue and grey, with soft, white clouds. The overall scene has a cinematic, slightly surreal quality.

DL for ENG

## LECTURE 5.1

# Convolutional and Graph Networks

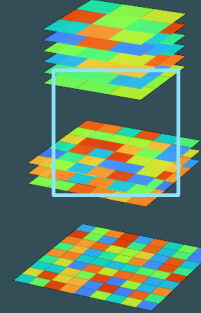
**Remus Teodorescu**

*With support from Research Assistant Noman Khan -  
nomank@energy.aau.dk*

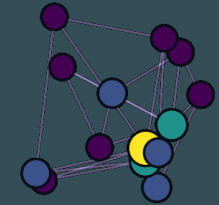
Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

## CONVOLUTION · GRAPH NETWORKS



a grid — every pixel has the same neighbours



a graph — whoever the edges connect

*A convolutional network reads an image through small windows and reuses the same kernel at every position, so its parameters do not grow with the image. A graph neural network does the same on a graph: every node updates from its neighbours with one shared rule, here on the six-bus network of the exercise.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 **Local rules** — why a dense layer is a poor fit for an image
- 2 **Convolution** — what a convolutional network is, and its main pieces
- 3 **Graphs** — the same idea on data made of connected parts

*One small example runs through the lecture, a six-bus power network. You work with it again in the exercises.*

## READ, AND WHAT EACH ANSWERS

|              |                                                     |
|--------------|-----------------------------------------------------|
| UDL ch. 10.1 | why is a dense layer a poor fit for an image?       |
| UDL ch. 10   | what does a convolution compute, and how is it set? |
| UDL ch. 13   | how does the same idea work on a graph?             |

# Recap

## 1 – 2 · DEEP NETWORK

### 1 What is a deep network, and what does depth give that width does not?

Shallow networks composed: the output of one layer is the input of the next. Each layer folds the input again, so the linear regions multiply with depth where width only adds them: more regions for the same number of parameters. *UDL ch. 4.1 – 4.5; Ex\_04 notebook 04*

### 2 What made deep networks trainable, and how is a first one sized?

The gradient reaches the first layers as a product of many factors and can vanish on the way; ReLU and He initialisation keep it alive. Start at four layers of 32 neurons, and change the size only when the held-out error says so. *UDL ch. 7.5, 11; Ex\_04 notebook 04*

## 3 – 4 · GENERALISATION

### 3 What is generalisation, and how do training and held-out error behave as capacity grows?

A low error on data the model has not seen. The training error falls all the way as capacity grows; the held-out error falls, then rises once the model starts to fit the noise. A fit through every point with a poor held-out error is an overfit. *UDL ch. 8.1 – 8.2; Ex\_04 notebook 05*

### 4 How is a model chosen, and how is it kept from overfitting?

With three sets: train on the first, choose the capacity and the settings on the validation set, and report the test set once, at the end. Regularise before you shrink the network: weight decay and early stopping are the two methods to try first. *UDL ch. 8.5, 9.1; Ex\_04 notebook 05*

# Local Rules

**Local rule** — one small calculation that looks only at a point and its neighbours, with the same weights reused at every point. On a grid it is a convolution; on a graph it is message passing.

## A DENSE LAYER ON AN IMAGE, AND WHAT REPLACES IT

- a **256 × 256 image** — 65,536 inputs, one per pixel
- a **dense layer of 1,000 neurons** — about 65 million weights
- the **problem** — to it, the pixels have no neighbours
- the **new rule** — a small window, the same weights everywhere
- one **3 × 3 kernel** — 10 parameters, for any image size
- the **lesson** — build in what we know about the data

### ON A GRID — CONVOLUTION

|                   |                                          |
|-------------------|------------------------------------------|
| <b>computes</b>   | a weighted sum over a small window       |
| <b>parameters</b> | the kernel weights and one bias          |
| <b>reused</b>     | at every position, so for any image size |

### ON A GRAPH — MESSAGE PASSING

|                   |                                      |
|-------------------|--------------------------------------|
| <b>computes</b>   | a sum over the connected nodes       |
| <b>parameters</b> | one weight matrix and one bias       |
| <b>reused</b>     | at every node, so for any graph size |

# Convolutional Network

**Convolutional network** — a network for data laid out on a grid, such as an image. Each layer looks at small neighbourhoods and reuses the same weights at every position.

## KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a feed-forward network whose layers are convolutions
- **function** — image in, features or label out, shift-equivariant
- **architecture** — convolution and pooling, then a dense head
- **applications** — classification, segmentation, detection, 1-D signals

## AGAINST A DENSE NETWORK

|                 | dense network               | convolutional                        |
|-----------------|-----------------------------|--------------------------------------|
| connections     | every input to every neuron | a small window only                  |
| weights         | one per connection          | one kernel, reused at every position |
| parameters      | grow with the input size    | weights and biases, any image size   |
| a shifted input | an unrelated output         | the same shift of the output         |
| input size      | fixed when designed         | any, with global pooling             |

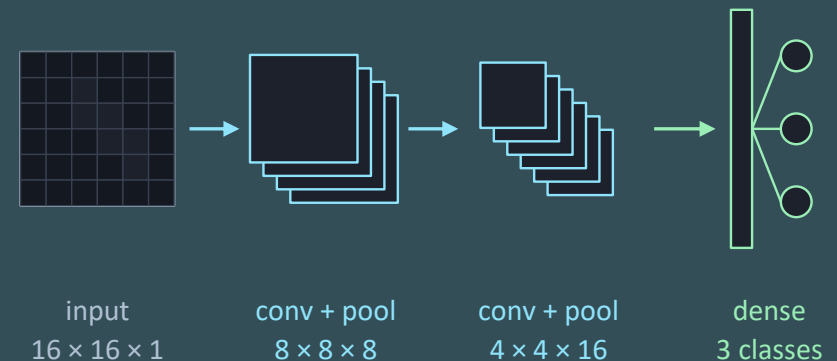
## CONVOLUTION AND NETWORK

$$h_{ij} = a \left( \sum_m \sum_n k_{mn} x_{i+m, j+n} + b \right)$$

$x$  the input image,  $k_{mn}$  the kernel,  $b$  one bias  
 $a$  the activation,  $h_{ij}$  one output pixel

### CONVOLUTION IN PYTORCH

```
conv = nn.Conv2d(1, 8, 3) # eight 3 × 3 kernels
y = conv(x)               # same weights everywhere
```



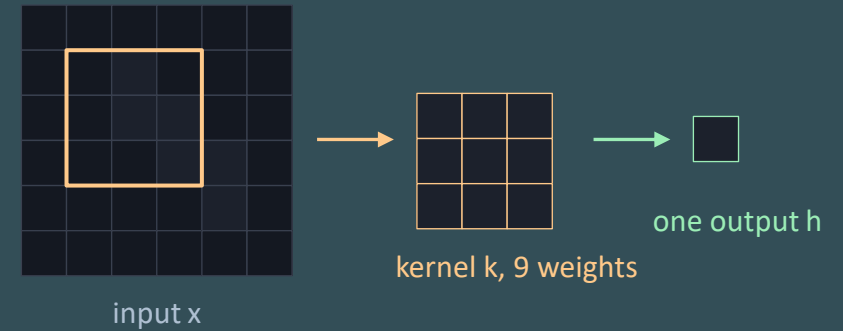
# Kernel

**Kernel** — the small set of weights a convolution slides over its input. At each position it takes a weighted sum of the neighbouring values, adds a bias and applies the activation.

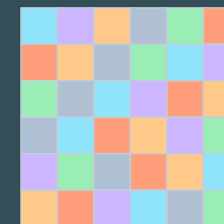
## A WEIGHTED SUM OVER A SMALL WINDOW

- a **3 × 3 window** — multiply, sum, add a bias, apply the activation
- **move one pixel and repeat** — with the same nine weights
- **the nine weights are learned** — once set by hand, now trained
- **the output** — an image of the same kind, one value per position
- **an edge detector** — weights -1, 0, +1 across the columns

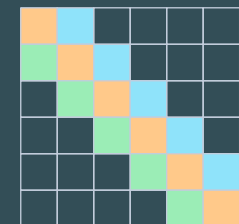
## ONE WINDOW, ONE OUTPUT PIXEL



## THE SAME IDEA AS A WEIGHT MATRIX



dense  
36 weights



convolution  
3 weights, reused

# Convolutional Network Hyperparametrization

**Hyperparameters** — the choices made before training that fix a convolution layer's shape: the kernel size, how far the kernel moves each step, how the border is filled, and how many feature maps go in and come out. The kernel weights are the parameters, learned in training.

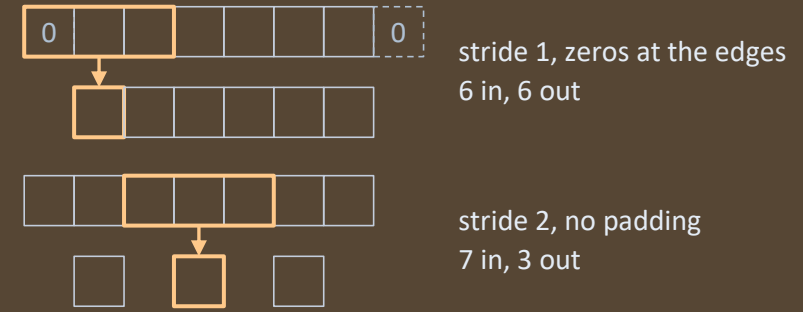
## CHOSEN BEFORE TRAINING, THEN LEARNED

- **stride** — the step the window takes; a step of 2 halves the image
- **padding** — a border of zeros, so the output keeps the input's size
- **channels** — feature maps, each with its own kernel
- **parameters** — kernel area × maps in × maps out, plus biases

$$P_{\text{dense}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}} \quad P_{\text{conv}} = K^2C_{\text{in}}C_{\text{out}} + C_{\text{out}}$$

$N_{\text{in}}, N_{\text{out}}$  neurons in and out,  $K$  the kernel side,  $C_{\text{in}}, C_{\text{out}}$  channels in and out

## STRIDE AND PADDING, IN 1D



## HYPERPARAMETRIZATION IN PYTORCH

```
conv = nn.Conv2d(in_channels=1, # grey image: 1 map in
                  out_channels=8, # 8 kernels, 8 maps out
                  kernel_size=3, # each kernel 3 x 3
                  stride=1,      # move one pixel a step
                  padding=1)     # zero border: 16x16 kept
pool = nn.MaxPool2d(2)          # halve each side: 16 -> 8
# learned: 3*3*1*8 weights + 8 biases = 80 parameters
```

# Pooling and Hierarchy

**Pooling** — a step that shrinks a feature map by summarising each small block, for example by its largest value. Stacked with convolutions, it builds features from edges up to whole objects.

## SHRINK THE IMAGE, BUILD UP FEATURES

- **pooling** — each  $2 \times 2$  block becomes its largest value
- **why pool** — less computation; small shifts matter less
- **image shrinks, channels grow** — less where, more what
- **the hierarchy** — edges, textures, parts, whole objects
- **nobody designs it** — it emerges from training
- **when not to pool** — when the answer must say where

### MAX POOLING, $2 \times 2$

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 0 |
| 4 | 2 | 1 | 1 |
| 0 | 1 | 5 | 2 |
| 2 | 0 | 3 | 4 |

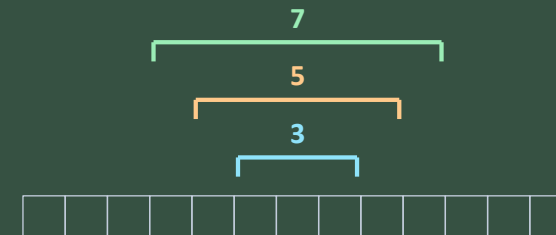
input,  $4 \times 4$ 

|   |   |
|---|---|
| 4 | 2 |
| 2 | 5 |

output, largest of each

### WHAT ONE NEURON SEES, BY DEPTH

*each layer widens the reach*



layer 3 · parts

layer 2 · textures

layer 1 · edges

# Equivariance

**Equivariance** — shift the input and the output shifts the same way.  
 Invariance: shift the input and the output does not change. Convolution is equivariant; pooling adds invariance.

## EQUIVARIANT AND INVARIANT

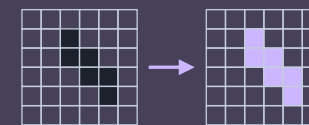
- **equivariant** — move the input, and the output moves with it
- **invariant** — move the input, and the answer stays the same
- **for example** — the crack map moves; “crack: yes” does not
- **convolution** — equivariant: the same kernel everywhere
- **pooling** — adds invariance by throwing position away

**TALK · 2 MIN** How do you understand convolution — what does it do to an image, in your own words? Then, in WHAT EACH OPERATION GIVES on the right, which row describes a crack detector that must say where the crack is, and which a classifier that only says whether there is one?

## WHAT EACH OPERATION GIVES

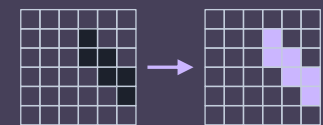
|                | property         | used for            |
|----------------|------------------|---------------------|
| convolution    | equivariant      | every layer         |
| pooling        | partly invariant | classification      |
| global average | invariant        | one label per image |
| no pooling     | equivariant      | segmentation        |

input, and its crack map



crack: yes

moved one pixel right



crack: yes

*the map moves: equivariant · the answer stays: invariant*

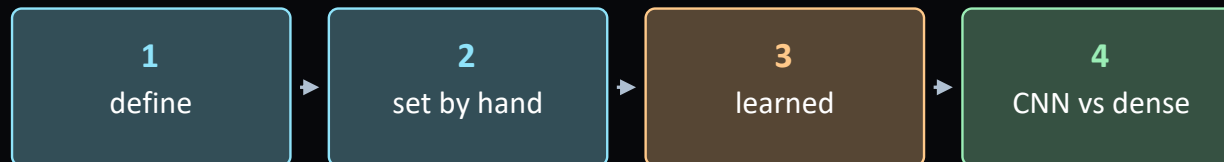
# Convolution in Practice

**A convolutional network in practice** — a few convolution and pooling layers, then a dense head, trained on labelled images. In Ex05 notebook 01: weld radiographs of clean plate, cracks and pits.

## WHAT THE CONVOLUTION SECTION SHOWED

- **local and shared** — small window, same weights everywhere
- **few parameters** — kernel weights and biases, for any image size
- **settings** — stride, padding and channels, chosen before training
- **pooling** — shrinks the image; deeper layers see more of it
- **equivariant** — output follows input; pooling adds invariance

## IN EX05 NOTEBOOK 01



## NOTEBOOK 01, THE RESULT

|               | parameters | train | held out |
|---------------|------------|-------|----------|
| convolutional | 2,019      | 1.000 | 0.989    |
| dense         | 16,643     | 0.995 | 0.711    |

*600 weld radiographs, 16 × 16 · clean, crack, pit*

## THE SAME RESULT IN UDL

**convolutional**      about 17 % test error

**dense**      about 40 % test error

*MNIST-1D · same depth and width · structure built in wins*

# Graphs

**Graph** — a set of nodes joined by edges. The nodes carry values, the edges say which nodes are connected (and may carry values too), and there is no grid and no natural order.

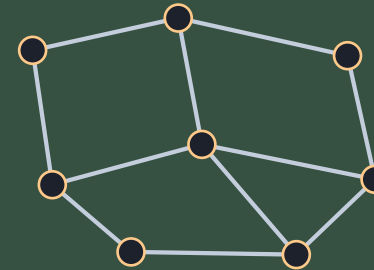
## NODES AND EDGES, IN FOUR FIELDS

|                    | nodes     | edges              |
|--------------------|-----------|--------------------|
| road map           | junctions | roads between them |
| molecule           | atoms     | chemical bonds     |
| electrical circuit | junctions | the components     |
| power grid         | buses     | transmission lines |

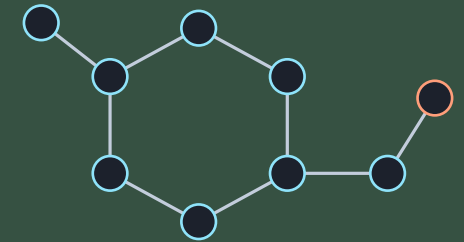
## THE TYPES WE USE

- **undirected** — a line carries power both ways
- **directed** — one way only, like one paper citing another
- **edge features** — numbers on each edge, like line admittance

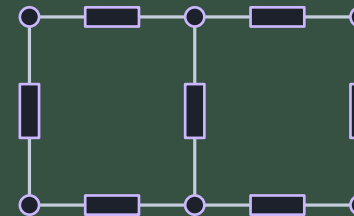
### FOUR GRAPHS



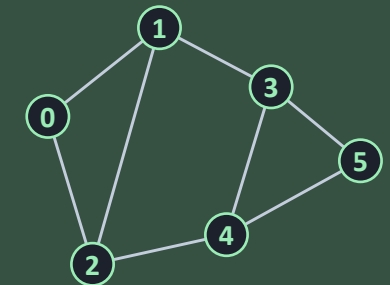
road map



molecule



electrical circuit



power grid, six buses

# Graph Representation

**Graph representation** — the encoding of a graph for computation: typically an adjacency matrix or edge list defining the connectivity, together with a node feature matrix and, optionally, an edge feature matrix.

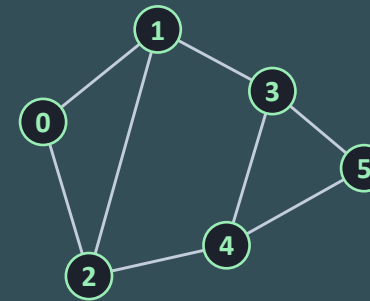
## THREE MATRICES DESCRIBE THE WHOLE GRAPH

### GRAPH REPRESENTATION IN PYTORCH

```
X = torch.tensor(bus_features) # (6, 6): a row per bus
E = torch.tensor(line_RX)      # (8, 2): a row per line, R and X
A = torch.zeros(6, 6)         # adjacency, bus by bus
for i, j in lines:             # each of the 8 lines
    A[i, j] = A[j, i] = 1      # both ways: undirected
# sparse: 16 of the 36 entries of A are 1
```

**TALK • 2 MIN** How would you explain a graph to someone outside engineering, and what makes a power grid one? Then, in the adjacency matrix on the right, which row belongs to bus 2, and which three columns of it are lit?

### THE SIX BUSES AS MATRICES



|   | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |

A, 6 × 6

|   | P | Q | V | S | PV | PQ |
|---|---|---|---|---|----|----|
| 0 |   |   |   |   |    |    |
| 1 |   |   |   |   |    |    |
| 2 |   |   |   |   |    |    |
| 3 |   |   |   |   |    |    |
| 4 |   |   |   |   |    |    |
| 5 |   |   |   |   |    |    |

X, 6 × 6

|     |      |
|-----|------|
| 0-1 | 16.5 |
| 0-2 | 14.2 |
| 1-2 | 16.5 |
| 1-3 | 12.4 |
| 2-4 | 14.2 |
| 3-4 | 16.5 |
| 3-5 | 9.9  |
| 4-5 | 12.4 |

E, 8 × 1, |y|

# Graph Neural Network

**Graph neural network** — its input is a graph. Each layer updates every node from its own value and its neighbours' (message passing). It is invariant to node count, numbering and topology changes.

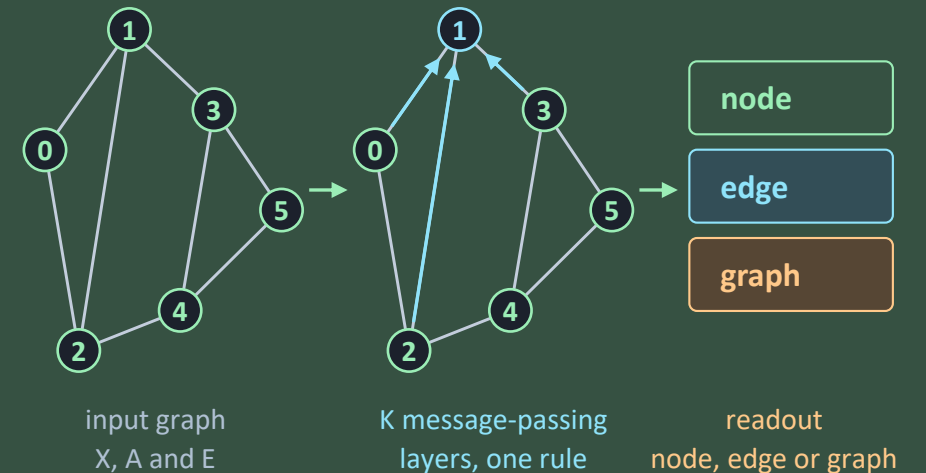
## KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — layers that update every node from its neighbours
- **function** — a graph in, X, A and E; an answer out
- **architecture** — K message-passing layers, then a readout
- **applications** — power grids, circuits, molecules, road maps

## AGAINST A DENSE NETWORK

|                  | dense network               | graph network                      |
|------------------|-----------------------------|------------------------------------|
| input            | one vector, fixed length    | a graph of any size, with features |
| connections      | every input to every neuron | along the edges only               |
| weights          | one per connection          | one rule, shared by every node     |
| renumbered nodes | an unrelated output         | the same output, renumbered        |
| topology         | fixed when trained          | an input; one model, many graphs   |

## THE ARCHITECTURE, ON THE SIX BUSES



*bus 1 gathers from buses 0, 2 and 3 · every bus does the same, with the same rule*

# Graph Tasks and Losses

**Graph tasks and losses** — a prediction for each node (bus voltage or angle), for each edge (line fault), or one for the whole graph (stability), trained with a regression or classification loss function as appropriate.

## WHAT IS PREDICTED, AND HOW IT IS SCORED

| task                 | output                 | loss                                 |
|----------------------|------------------------|--------------------------------------|
| node regression      | a number per node      | squared error, as in notebook 03     |
| node classification  | a class per node       | cross-entropy                        |
| edge prediction      | a probability per edge | binary cross-entropy                 |
| graph classification | one class per graph    | cross-entropy, on the averaged nodes |

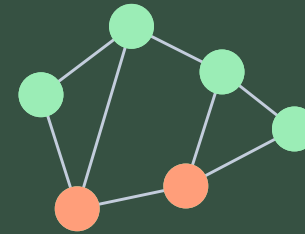
$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}$$

$N$  nodes,  $C$  classes,  $y_{ic} = 1$  for the true class,  $\hat{y}_{ic}$  the predicted probability

### GRAPH LOSSES IN PYTORCH

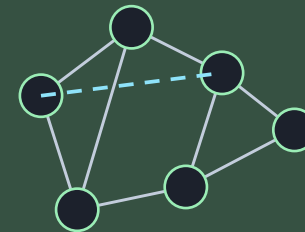
```
loss = F.mse_loss(out[:, 0], v)      # node regression: a voltage per bus
loss = F.cross_entropy(logits, labels) # a class per node, or per graph
g = out.mean(0)                      # a graph answer: average the nodes
```

## THREE LEVELS OF ANSWER



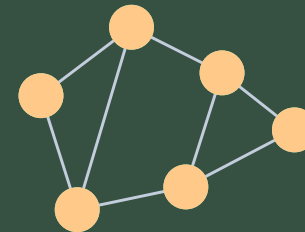
### node

a value at every bus, as in notebook 03



### edge

a probability for each line, such as whether it is present



### graph

one answer for the whole grid, such as whether it is stable

# Message Passing

**Message passing** — one layer of a graph network. Every node collects messages from its neighbours, aggregates them (sum, mean or max), and combines the result with its own state to get its new state. Each layer extends a node's reach by one hop, so the number of layers needed is related to the graph diameter, the longest shortest path between any two nodes.

## HYPERPARAMETERS AND DIAMETER IN PYTORCH

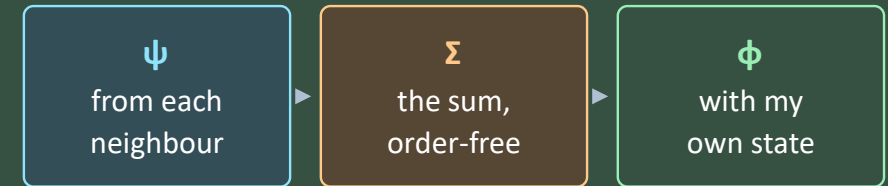
```
A1 = torch.eye(6) + A          # a bus and its neighbours
R, hops = A1, 1                # who has heard whom, after 1 hop
while (R == 0).any():           # someone not reached yet?
    R, hops = R @ A1, hops + 1  # one more hop
layers = hops                   # the diameter: 3 on the six buses
hidden, agg = 32, "sum"         # numbers per bus; sum, mean or max
act = nn.Tanh()                 # the activation in every layer
```

$$\mathbf{h}_{i(l+1)} = \phi \left( \mathbf{h}_{i(l)}, \sum_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_{i(l)}, \mathbf{h}_{j(l)}) \right)$$

$\mathbf{h}_i^{(l)}$  node  $i$  at layer  $l$ ,  $\mathcal{N}(i)$  its neighbours,  $\psi$  the message,  $\phi$  the combine

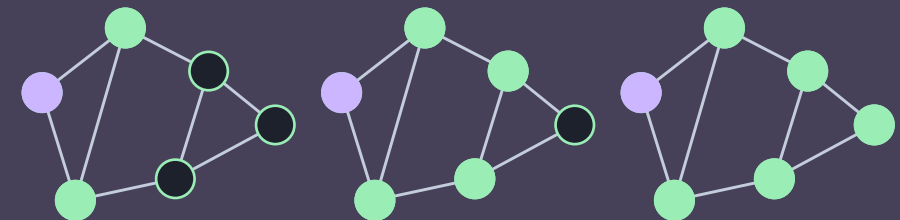
*the physics sets the depth, weights the edges, gives the ground truth*

## ONE NODE'S UPDATE



*same functions at every node · repeat for more hops*

## HOW FAR BUS 0 HEARS, LAYER BY LAYER



layer 1: 3 of 6 buses

layer 2: 5 of 6 buses

layer 3: 6 of 6 buses

*diameter 3: three layers reach every bus*

# Permutation Equivariance

**Permutation equivariance** — renumber the nodes and the node outputs are renumbered the same way; a graph-level output does not change at all (invariance). A GNN has this property by construction, because every node uses the same update and an order-independent aggregation.

## RENUMBER THE NODES, THE OUTPUT FOLLOWS

- **bus numbers** — bookkeeping, with no natural order
- **a graph network** — the output follows exactly
- **a dense network** — no such guarantee

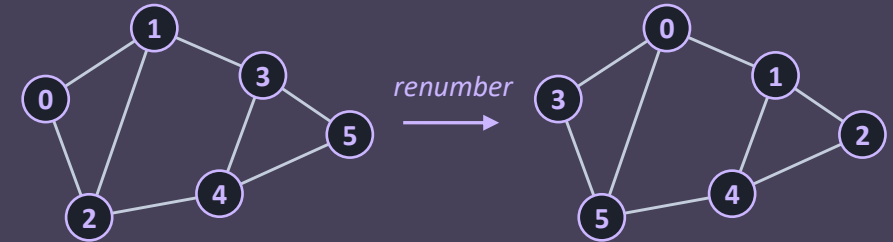
$$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^T) = \mathbf{P} f(\mathbf{X}, \mathbf{A})$$

$\mathbf{X}$  node features,  $\mathbf{A}$  the adjacency,  $\mathbf{P}$  any relabelling of the nodes,  $f$  the network

### RENUMBERING, CHECKED

```
perm = torch.randperm(6)           # renumber the six buses
out = gnn(x[perm], A[perm][:, perm]) # the same grid, relabelled
torch.allclose(out, gnn(x, A)[perm]) # True: the output follows
```

## ONE GRID, TWO NUMBERINGS



*same lines, same physics · only the labels changed*

## GRAPH AND DENSE, RENUMBERED

|                | graph network          | dense network         |
|----------------|------------------------|-----------------------|
| renumber input | output renumbered      | a different answer    |
| because        | one shared rule, a sum | a weight per position |
| input size     | any number of buses    | fixed when trained    |

# Advanced Topics

## RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic               | what it adds                       | read         | where it makes a difference |
|---------------------|------------------------------------|--------------|-----------------------------|
| residual networks   | skips let very deep networks train | UDL ch. 11   | image recognition at scale  |
| batch normalisation | rescales outputs between layers    | UDL ch. 11.4 | very deep image networks    |
| graph convolution   | neighbours weighted by degree      | UDL ch. 13.4 | citation, social networks   |
| graph attention     | learns how much a neighbour counts | UDL ch. 13.8 | molecules, recommendations  |
| encoder–decoder     | shrinks, then regrows the image    | UDL ch. 10.5 | medical image segmentation  |
| inductive learning  | predicts on graphs never seen      | UDL ch. 13.6 | new molecules, new networks |

### WHY THEY ARE LEFT OUT

Each of these earns its place on very large data sets: millions of photographs, molecules or social-network users. The problems in this course are small and physical, and today's convolution and message passing are enough for them.

# Exercise 5

## THE NOTEBOOKS, IN ORDER

- **01** — weld radiographs, clean, crack or pit: set kernels by hand, train a CNN, compare it with a dense network
- **02** — the six-bus grid as a graph: adjacency, neighbours, message passing, and the renumbering check
- **03** — a graph network learns the AC power flow: compared with a dense network, timed, and a line tripped
- **04** — after L5.2: persistence, a recurrent network and an LSTM forecast the hourly load
- **05** — the report: every notebook's answers, gathered, and one question to conclude

## THE QUESTION EACH NOTEBOOK ANSWERS

- 01** can a CNN beat a dense network?
- 02** how does a node hear its neighbours?
- 03** how close to AC power flow, how fast?

## REPORT QUESTION, TO CONCLUDE

Across a grid, a graph and a sequence: when did building the data's structure into the network beat a dense network, and what did it cost?

# Key Takeaways

## CONVOLUTIONAL NETWORK

|                    |                                                           |
|--------------------|-----------------------------------------------------------|
| <b>what it is</b>  | a network for grid data: small kernels, shared everywhere |
| <b>function</b>    | an image in; features, a label or a map out               |
| <b>train, tune</b> | Adam on a loss; kernel size, stride, padding, channels    |
| <b>vs dense</b>    | far fewer parameters, and it generalised: 0.99 vs 0.71    |
| <b>used for</b>    | inspection images, 1-D signals, segmentation, detection   |

## GRAPH NETWORK

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| <b>what it is</b>  | a network on a graph: nodes updated from their neighbours    |
| <b>function</b>    | a graph in; an answer per node, per edge or per graph        |
| <b>train, tune</b> | Adam on a loss; layers from the diameter, width, aggregation |
| <b>vs dense</b>    | as accurate here, any size, and it survives renumbering      |
| <b>used for</b>    | power grids, heating and gas networks, batteries, molecules  |

*Next — L5.2: RNN and LSTM.*

# Questions

## 1 – 2 · CONVOLUTIONAL NETWORK

### 1 What is a convolutional network, and what function does it implement?

A network for data on a grid. A small kernel of shared weights is moved across the image and computes the same weighted sum at every position; stacked layers and pooling turn the image into features, then into a label or a map. *UDL ch. 10.1 – 10.4; Ex\_05 notebook 01*

### 2 How does it differ from a dense network, and why does it do better on images it has not seen?

It has far fewer parameters, because one kernel is shared by all positions, and a feature learned in one place is found in every other: shifting the image shifts the output. In notebook 01 it scored 0.99 on unseen images against 0.71 for the dense network. *UDL ch. 10.2; Ex\_05 notebook 01*

## 3 – 4 · GRAPH NETWORK

### 3 What is a graph network, and what does one message-passing layer do?

A network on nodes and edges. In each layer every node collects the features of its neighbours, combines them with its own through shared weights, and is updated. After k layers a node has heard from every node within k edges. *UDL ch. 13.1 – 13.4; Ex\_05 notebook 02*

### 4 How does a graph network differ from a dense network, and why does that suit a power grid?

Its weights belong to the rule, not to the nodes, so the same network runs on a graph of any size and gives the same answer however the nodes are numbered. A tripped line is one edge removed, and nothing has to be trained again. *UDL ch. 13.2 – 13.6; Ex\_05 notebook 03*

NEXT

## L5.2 — RNN and LSTM

---

Today's structures were spatial, a grid and a graph. Next time the structure is temporal: how to carry information from the start of a long sequence to its end.

### BEFORE THE NEXT SESSION

**read** GBC ch. 10.1 – 10.2, 10.10

**run** Ex\_05 notebooks 00 to 03

**bring** your tripped-line result

**think about** layer count against graph diameter

*both networks' errors, intact grid and after the trip, and the speed*